

```

In [16]: #Graph representation
n=6 #Number of nodes (vertices)
e=7 #number of edges

edges=[(0,1),(0,3),(0,4),(1,2),(1,5),(2,4),(3,4)] #List of Tuple

#Adjacency List
adjList=[]

for i in range(n):
    adjList.append([])

for edge in edges:
    x=edge[0]
    y=edge[1]

    adjList[x].append(y)
    adjList[y].append(x)
for i in range(n-1):
    print(f'{i}-->{adjList[i]}')

print()
#Adjacency matrix
adjMatrix=[]
for i in range(e):
    adjMatrix.append([-1]*n)
print(adjMatrix)
print()

for edge in edges:
    x=edge[0]
    y=edge[1]

    adjMatrix[x][y]=1
    adjMatrix[y][x]=1
for i in range(n):
    print(f'{adjMatrix[i]}')

```

```
0-->[1, 3, 4]
```

```
1-->[0, 2, 5]
```

```
2-->[1, 4]
```

```
3-->[0, 4]
```

```
4-->[0, 2, 3]
```

```

[[-1, -1, -1, -1, -1, -1], [-1, -1, -1, -1, -1, -1], [-1, -1, -1, -1, -1, -1], [-1, -1, -1, -1, -1, -1], [-1, -1, -1, -1, -1, -1], [-1, -1, -1, -1, -1, -1]]

```

```
[-1, 1, -1, 1, 1, -1]
```

```
[1, -1, 1, -1, -1, 1]
```

```
[-1, 1, -1, -1, 1, -1]
```

```
[1, -1, -1, -1, 1, -1]
```

```
[1, -1, 1, 1, -1, -1]
```

```
[-1, 1, -1, -1, -1, -1]
```

```

In [ ]: #Breadth First Search-->Implements using queue
from collections import deque
class Queue:

```

```

def __init__(self,capacity):
    self.capacity=capacity
    self.q=deque()

def isEmpty(self):
    return len(self.q)==0
def isFull(self):
    return len(self.q)==self.capacity

def enqueue(self,x):
    if self.isFull():
        return f"Queue OverFlow"
    else:
        self.q.append(x)
        return f"{x} is inserted"
def dequeue(self):
    if self.isEmpty():
        return f'Queue Underflow'
    else:

        return self.q.popleft()

def front(self):
    if self.isEmpty():
        return f"Queue underflow"
    else:
        return f"{self.q[0]} is the front element"
def size(self):
    return len(self.q)

def rear(self):
    if self.isEmpty():
        return f"Queue underflow"
    else:
        return f"{self.q[-1]} is the rear element"

def display(self):
    if self.isEmpty():
        return f"Queue Underflow"
    return self.q

n=6 #Number of nodes (vertices)
e=7 #number of edges

edges=[(0,1),(0,3),(0,4),(1,2),(1,5),(2,4),(3,4)] #List of Tuple

#Adjacency List
adjList=[]

for i in range(n):
    adjList.append([])

for edge in edges:
    x=edge[0]
    y=edge[1]

    adjList[x].append(y)
    adjList[y].append(x)
for i in range(n):
    print(f'{i}-->{adjList[i]}')

```

```

#BFS
q=Queue(20)
visited=[False]*n
ans=[]

q.enqueue(0)
visited[0]=True
# ans.append(0)

while q.size()>0:
    front=q.dequeue()
    ans.append(front)
    for neighbour in adjList[front]:
        if visited[neighbour]==False:
            visited[neighbour]=True
            q.enqueue(neighbour)

print("BFS Traversal :", ans)

```

```

0-->[1, 3, 4]
1-->[0, 2, 5]
2-->[1, 4]
3-->[0, 4]
4-->[0, 2, 3]
5-->[1]
BFS Traversal : [0, 1, 3, 4, 2, 5]

```

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click [here](https://aka.ms/vscodeJupyterKernelCrash) for more info.

View Jupyter [log](command:jupyter.viewOutput) for further details.

```

In [ ]: #Depth First Tree->Implement using stack
class Stack:
    def __init__(self,capacity):
        self.capacity=capacity
        self.st=[]

    def isEmpty(self):
        return len(self.st)==0

    def isFull(self):
        return len(self.st)==self.capacity

    def push(self,x):
        if self.isFull():
            return f"Stack Overflow"
        self.st.append(x)

    def pop(self):
        if self.isEmpty():
            return "Stack Underflow"
        return self.st.pop()

    def top(self):

```

```

        if self.isEmpty():
            return "Stack Underflow"
        else:
            return self.st[-1]

    def size(self):
        return len(self.st)

    def display(self):
        if self.isEmpty():
            return "Stack underflow"
        else:
            return self.st

#Adjacency list
n=6
e=7

edges=[(0,1),(0,3),(0,4),(1,2),(1,5),(2,4),(3,4)]

adjList=[]

for i in range(n):
    adjList.append([])

for edge in edges:
    x=edge[0]
    y=edge[1]

    adjList[x].append(y)
    adjList[y].append(x)
for i in range(n):
    print(f'{i}-->{adjList[i]}')

#DFS

st=Stack(20)
visited=[False]*n
ans=[]

st.push(0)

while st.size()>0:
    node=st.pop()

    if visited[node]==False:
        visited[node]=True
        ans.append(node)

        for neighbour in reversed(adjList[node]):
            if visited[neighbour]==False:
                st.push(neighbour)
print("DFS Traversal :", ans)

```

```
0-->[1, 3, 4]
1-->[0, 2, 5]
2-->[1, 4]
3-->[0, 4]
4-->[0, 2, 3]
5-->[1]
DFS Traversal : [0, 1, 2, 4, 3, 5]
```